2025 MITRE eCTF
Design Document

Team UCCS 1

University of Colorado Colorado Springs

Table of Contents

1 Purpose

Our team will design and implement a Satellite TV System solution. The system will securely encode and decode satellite TV data streams while protecting against unauthorized access to protected channels. Our team will design both the Encoder that will encode TV frames to be broadcast over the satellite in real-time to all teams as well as the hardware Decoder responsible for securely decoding frames for display on the TV.

2 Assets

AST-1: Decoder

DEC-1 Decoder ID

DEC-2 Decoder Global Secrets

- Subscription Package Key,
- Authentication Verification Key,
- Emergency Channel Key

DEC-3 Decoder Stored Subscriptions

DEC-4 Decoder device time

AST-2: Subscription Package

SUB-1 Subscription Update

SUB-2 Subscription Update Decoder ID

SUB-3 Subscription Update Channel Number

SUB-4 Subscription Start

SUB-5 Subscription End

SUB-6 Channel Frame Decode Key

AST-3: Frames

FRM-1 Frame Channel

FRM-2 Frame Timestamp

FRM-3 Frame Data

AST-4: Satellite

SAT-1 Satellite Signal

3 Functional Requirements

FR-1: Build Requirements

FR-1.1 Build Environment
The build environment must install all dependencies, compilers, and tool required for building the decoder.

FR-1.2 Build Deployment
A deployment is defined as an Encoder and Decoder set using the same set of global secrets. A deployment must generate any required gloabl secrets

FR-1.3 Build Decoder
The decoder must be built using the docker container constructed during the enviroment build and will use the MITRE provided host tool.

FR-2: Encoder

FR-2.1 Encoder
The Encoder must function together with the Uplink and Satellite system. The Encoder must accept raw frame data, channel numbers, and timestamps and must produce encoded frames.

FR-3: Decoder

FR-3.1 List Channels
The Decoder must be able to list the channel numbers that the Decoder has a valid subscription for.

FR-3.2 Update Subscription
The Decoder must be able to update it's channel subscriptions with a valid update package. If a subscription update for a channel is received for a channel that already has a subscription, the Decoder must only use the latest subscription update.

FR-3.3 Decode Frames
d The Decoder must properly decode TV frame data from any channel that has a valid and active subscription installed or for any emergency broadcast frames.

4 Non-Functional

NFR-1: Timing Requirements

- NFR-1.1 The frames that need to be decoded should take no more than 150 ms to decode. This prevents stuttering and produces a consistent video or image.
- NFR-1.2 The host device should take a maximum of 1s to wake up when the decoder gives a prompt.
- NFR-1.3 Listing channels and updating subscriptions should not take more than 500 ms.

NFR-2: Throughput Requirements

- NFR-2.1 The encoder must encode at 1000 frames per second.
- NFR-2.2 The decoder must decode frames at 10 frames per second. This ensures the corresponding video quality is at the highest possible even after compression and transmission.

NFR-3: Component Size Requirement

- NFR-3.1 Channel and Decoder ID should be a 32-bit unsigned integer.
- NFR-3.2 The timestamp should be a 64-bit unsigned integer.
- NFR-3.3 The decoded frame should have a maximum size of 64 bytes.

NFR-4: Channel Requirements

- NFR-4.1 The decoder should be able to store eight standard channels, with an additional channel as the emergency channel. This 9th channel (Emergency channel) should strictly be channel 0.

NFR-5: Additional Functional Requirement

- NFR-5.1 The host device will initiate all communications with the decoder except for debug messages.
- NFR-5.2 All communications between the Decoder and the Host Computer will take place over UART at a baud rate of 115200.

5 Security Goals and Requirements

SG-1: A decoder must only decode TV frames whose timestamps fall withing the time range of a valid subscription.

SR-1.1 A decoder must verify that timestamps of incoming frames are within the range specified by the subscription for that channel

SG-2: A decoder must only decode TV frames with monotonically increasing timestamps.

SR-2.1 A decoder must reject any frame received which violates increasing timestamp order.

SG-3: A decoder must only decode TV frames generated by the satellite system the decoder was provisioned for.

SR-3.1 A decoder must not be capable of decoding frames not generated by the satellite system it was provisioned for.

SG-4: A decoder must only accept subscription updates generated by the satellite system the decoder was provisioned for.

SR-4.1 A decoder must verify the authenticity of subscription update packages and reject updates not originating from the satellite system.

SG-5: A decoder must only accept subscription updates generated for that decoder.

SR-5.1 A decoder must verify that a subscription update is intended for that decoder and reject any for other decoders.

6 Cryptography

We are using Monocypher library for symmetric and asymmetric encryptions. The host will generate global secrets, which will be deployed on all of the decoders.

Monocypher is a modern, small, and easy-to-use cryptographic library designed to provide secure, efficient cryptographic primitives. It serves as a lightweight alternative to larger libraries like libsodium or OpenSSL but with a simpler API and a focus on security. It is:

- Small. Sloccount counts under 2000 lines of code, small enough to allow audits. The binaries can be under 50KB, small enough for many embedded targets.
- Easy to deploy. Just add monocypher.c and monocypher.h to the project. They compile as C99 or C++ and are dedicated to the public domain (CC0-1.0, alternatively 2-clause BSD).
- Portable. There are no dependencies, not even on libc.
- Honest. The API is small, consistent, and cannot fail on correct input.
- Direct. The abstractions are minimal. A developer with experience in applied cryptography can be productive in minutes.
- Fast. The primitives are fast to begin with and performance wasn't needlessly sacrificed. Monocypher holds up pretty well against libsodium, despite being closer in size to TweetNaCl (see the more detailed benchmark).

More details about the system design can be found in Sec.8. We are using *generate_key* function from Monocypher which has a header like:

```
generate_key(length=None, method=None) -> 'bytes'
```

This function generate a random key which:

- By default has a key length of 32 bytes.
- By default, it applies *ChaCha20* cipher to Python's random number generator to increase entropy (does not improve randomness).
- The returned key is as secure as the random number generator of Python. More details on the randomness can be found here: Python secrets module.

We are using *generate_signing_key_pair* function from Monocypher for signing and verification purposes. It has a header like this:

```
generate_signing_key_pair() -> 'tuple[bytes, bytes]'
```

This function generate a new keypair for signing using default settings. To print a key, use the following code snippet:

```
import binascii
print(binascii.hexlify(key))
```

7 Threat Identification and Mapping to Security Requirements

We use an approach that provides a systematic way of identifying threats and assets as a tuple consisting of three parts: action, asset, harm. The resulting tuple is used to formulate statements of the type: action X on/to asset Y could cause harm Z.

Details and Justifications

T-1: Leaking the Global Secrets will allow the attacker to spoof subscription updates.

T-2: Tampering with the stored subscription in decoder will allow the attacker to decode frame packages without legitimate subscription update package.

T-3: Spoofing the subscription update package will allow the attacker to decode frame packages without legitimate subscription update package.

T-4: Tampering with the decoder device time or frame's timestamp will allow the attacker to decode frames using expired subscription.

T-5: Tampering with the decoder device ID will allow the attacker to load a subscription from a subscription update package intended for a different device.

T-6: Tampering with the subscription update package's decoder ID will allow the attacker to load a subscription intended for a different device.

T-7: Spoofing the subscription update package will allow the attacker to load illegitimate subscription.

T-8: Tampering with the frame timestamps will allow the attacker to replay saved frames.

T-9: Spoofing the satellite signal will allow the attacker to prevent neighbors from receiving the real satellite signal.

Threat Number	Action	Asset	Harm	Security Target Reference
T-1	Leaking	DEC-2	Spoof Subscription Updates	SR-4.1, SR-5.1
T-2	Tampering	DEC-3	Update stored, expired subscription without legitimate subscription update package	SR-3.1
T-3	Spoofing	SUB-1	Update stored, expired subscription without legitimate subscription update package	SR-3.1, SR-4.1, SR-5.1
T-4	Tampering	DEC-4 FRM-2	Decode frames using expired subscription	SR-1.1
T-5	Tampering	DEC-1	Allows loading a subscription intended for a different device	SR-5.1
T-6	Tampering	SUB-2	Allows loading a subscription intended for a different device	SR-5.1
T-7	Spoofing	SUB-1	Loading illegitimate subscription	SR-4.1
T-8	Tampering	FRM-2	Allows attacker to replay saved frames	SR-1.1, SR-2.1
T-9	Spoofing	SAT-1	Prevent Neighbor from receiving the real satellite signal	SR-3.1

Table 1

8 System Architecture Diagram

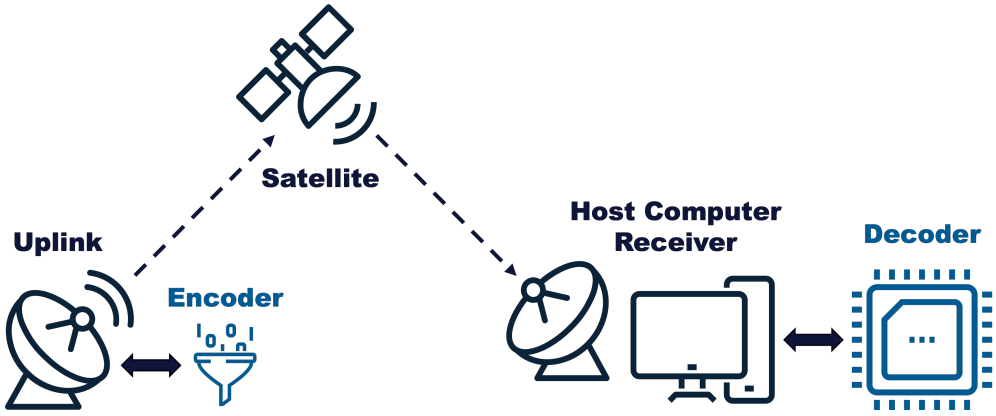


Figure 1: High Level System

Fig.1 is a high-level diagram of the system we are trying to implement. The host (encoder) will encode subscription update package and frame package and send them to the satellite to be broadcast to all the decoders. The host will also generate global secrets and deploy them on every decoder. The followings will demonstrate what our global secret will contain:

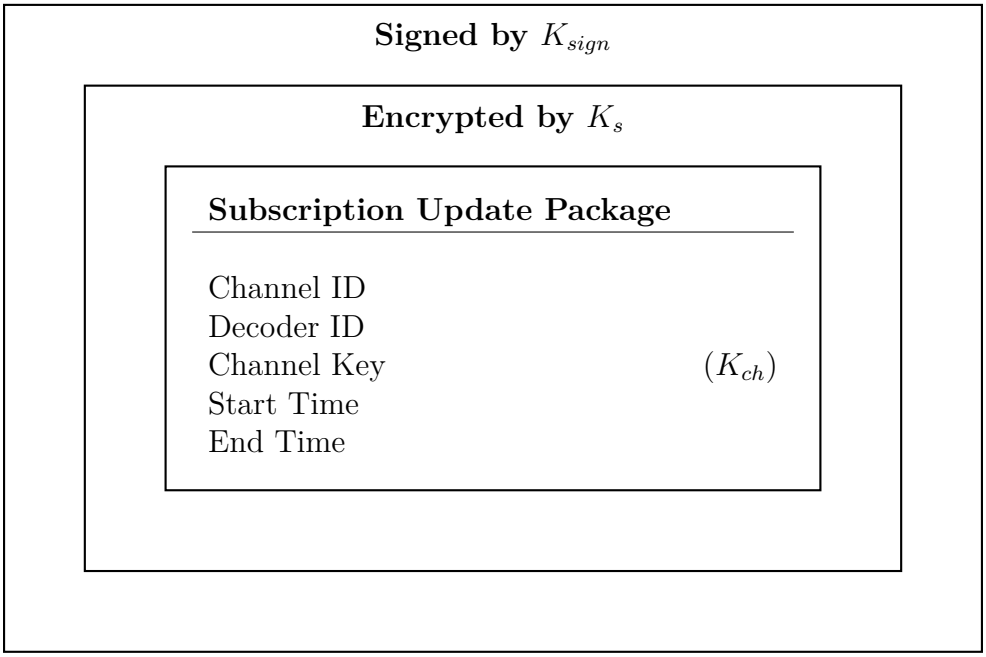
Global Secrets	
List of Channels	
Subscription Key	K_s
Signing Key	
Verification Key	K_{verif}
Emergency Channel Key	K_{C_0}
Other Channel Keys	K_{ch}

This contains all the secrets that will be shared among the host (encoder) and all of the decoders. Initially, the decoder will only contain the following secrets:

Global Secrets (Initial Decoder)	
List of Channels	
Subscription Key	K_s
Verification Key	K_{verif}
Emergency Channel Key	K_{C_0}

The subscription key will be distributed to all decoders, and is used to encrypt and decrypt subscription update packages so as to not lead the channel keys contained in the

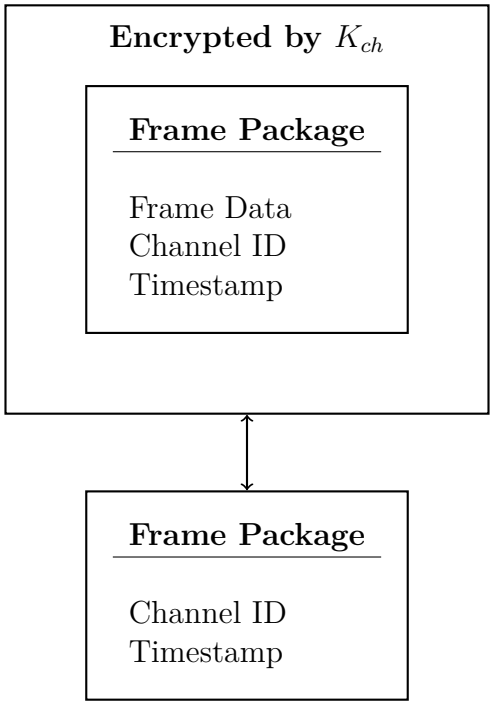
subscription updates. This is to prevent unknown decoders from obtaining the content of subscription update packages. The verification key is included to allow decoders to verify the authenticity of subscription update, ensuring that only subscription updates generated by the satellite system are accepted. The emergency channel key is included so that every decoder can access the emergency channel by default. We decided to include K_{C_0} to simplify the decoding processing, so that we won't need to implement a separate algorithm to decode frames from the emergency channel. The following shows what our subscription package will contain:



The channel ID is the number for which this subscription update is for (e.g. 0, 1, 2, …). Since every decoder will receive the broadcast from the satellite, it is necessary to include decoder ID in subscription update package so that only the decoder with matching ID will be able to obtain the new subscription. The channel key is used to decode upcoming frames from that channel. The subscription update package also contains start and end time for expiration checking.

Subscription update packages will first be encrypted using the subscription key (K_s) to ensure only valid encoders can update their subscriptions. On top of this layer of encryption, the subscription update package is encrypted again using the signing key (K_{sign}) that is only contained in the host. This is to ensure encoders only accepts valid packages from the satellite using K_{verif}, preventing pesky neighbors from interfering.

A typical frame package contains:



The frame data, timestamp, and channel ID are all encrypted using the channel key by the encoder. Un-encrypted Channel ID and Timestamp are passed along side the encrypted data for use in selecting the correct Channel Key and for initially verifying the Timestamp sequence. Once decryption has taken place, the decrypted Timestamp and Channel ID are checked against the plaintext version.

For subscription update package, our processing order is:

1. Decrypt using verification key (K_{verif}).
2. Decrypt using subscription update key (K_s).
3. Verify that the decrypted decoder ID matches the current decoder ID.
4. Store all other information (Channel ID, K_{ch}, Start and end time).

For frame package, our processing order is:

1. Use the decrypted channel ID to choose which K_{ch} to use.
2. decrypt using channel key (K_{ch}).
3. Check the timestamp is valid.
4. Deliver the frame data.

9 Assumptions

ASP-1: System Design

ASP-1.1 We are assuming the subscription updated key (K_s) is in the flash memory, and it would be hard for adversaries to access the flash and read or modify it. More information on: Sec.8.

ASP-1.2 We are assuming the Decoder ID is in the flash memory as well, which would be hard for the adversaries to modify it.

ASP-1.3 We are assuming the subscription key K_{ch} will be stored in flash memory.

ASP-1.4 We are assuming the previous frame package's timestamp will be stored in RAM.

10 Traceability Matrix

Security Requirement	Description	Assets	Status	Tests
SR-1.1	Frame timestamps must be within the range specified by the subscription.	SUB-4, SUB-5, FRM-2	100%	Passed
SR-2.1	Decoder must reject any frames violating increasing timestamp order.	DEC-4, FRM-2	100%	Passed
SR-3.1	Decoder must only decode frames generated by the satellite.	DEC-2, SAT-1, AST-3	100%	Passed
SR-4.1	Decoder must only accept subscription packages from the satellite.	DEC-2, SAT-1, AST-2	100%	Passed
SR-5.1	Decoder must only accept subscription packages intended for it, not other decoders.	DEC-1, SUB-2	100%	Passed

Table 2

11 Dependencies

- PyMonocypher: For the host (encoder), this python package uses Cython to wrap the Monocypher C library.

12 Change Log

Date	Details	Reference
01/30/2025	Added threat description in tuple format	Table 1
01/30/2025	Added initial set of assets	Asset list
02/04/2025	Updated the list of assets to be more detailed.	Asset list
02/04/2025	Updated the threat description to be more detailed	Table 1
02/06/2025	Updated the asset names to be more descriptive	Asset list
02/07/2025	Added functional requirements	Sec.3
02/10/2025	Added security requirements from the rules	Sec.4
02/17/2025	Added some system diagrams	Sec.8
02/17/2025	Added assumptions regarding system design	Sec.9
02/17/2025	Added Purpose	Sec.1
02/17/2025	Added cryptography tool	Sec.6
02/24/2025	Added monocypher and messagepack dependencies	Sec.11
02/24/2025	Updated cryptography to include how keys are generated	Sec.6
02/25/2025	Updated threat description details	Sec.7
03/13/2025	Refactored Security Goal/Requirement, Functional and Non-functional Requirements	Sec.5
03/17/2025	Added Security Requirements to Security Target Reference	Sec.7
03/17/2025	Updated Traceability Matrix	Sec.10